📌 Example 5: Regulatory QA using Local RAG 🧠📘

> **Synopsis**
>
> This example shows how **domain-specific questions** related to food safety, materials, or regulatory compliance can be answered using a **local LLM** connected to your own **Markdown-based Knowledge Base (KB)**.
> The approach leverages **RAG (Retrieval-Augmented Generation)** to simulate scientific or regulatory reasoning without needing internet access or external APIs.

🔑 Key Concepts

In this example, we explore how to:

- Build a **local knowledge base (KB)** using your own documentation (scientific, regulatory, experimental).
- Use **HuggingFace embeddings** to encode your KB for retrieval.
- Run a **local open-source LLM** (e.g. Mistral) using **Ollama** to answer technical questions.
- Perform **offline and confidential RAG**, avoiding any vendor lock-in.

This pipeline mimics how SFPPy can be integrated with LLMs to automate and audit **domain-specific reasoning** — a concept referred to as **SAG (Simulation-Augmented Generation)** in the *Generative Simulation* initiative.

📦 Required Dependencies (minimal)

```
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
```

- Ollama must be **installed and configured** on your machine.
- The KB should be located in `docs/KB/`, and a sample is provided as `KB.zip`.

> ⚠️ **Warning:** This example requires a specific installation (see the final *Installation* section), proper hardware configuration, and locally served LLM tools (e.g., via `ollama` or similar).
> The LLM and vector store tools are **not included** in SFPPy and must be installed separately.

📂 Load and Embed the Knowledge Base

```
documents = SimpleDirectoryReader(input_dir="./docs/KB", recursive=True).load_data()
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
index = VectorStoreIndex.from_documents(documents, embed_model=embed_model)
```

The output will be

```
📂 Building KB index from Markdown files...
🔍 Indexed documents: 66
```

🧠 Connect to Local LLM and Query

```
llm = Ollama(model="mistral")
query_engine = index.as_query_engine(llm=llm)

# Ask a question (RAG happens here)
response = query_engine.query("What is the migration of Irganox 1076?")
print(response)
```

✅ Output Sample

```
Irganox 1076 is a hindered phenolic antioxidant used in polyolefins and food packaging.
It can migrate into fatty foods and is regulated in the EU (SML = 6 mg/kg, FCM 902) and by the US
FDA.
SFPPy allows simulating its migration based on D, k, temperature, and geometry (e.g., in
example1.py).
```

✅ A full example with a generic question

You can append the following section to your `example5.md` to complete the documentation with a practical demonstration and performance note:

✅ A Full Example with a Generic Question

Once the knowledge base (KB) is indexed, you can ask generic questions such as:

```
response = query_engine.query("Explain what is diffusivity.")
print(response)
```

🔍 Indexed documents: 66 🔄 Index persisted to: `docs/KBIndex`

The answer will be:

> In the provided context, diffusivity refers to the rate at which a substance spreads or diffuses through a medium. The document discusses diffusion coefficients (Dw) for various organic and inorganic chemicals in water at 20°C, as well as methods to approximate diffusion coefficients for other materials like skin and soils. It's also mentioned that the partition coefficient (K), which can be useful in migration calculations, is related to the solubility of a substance in both the polymer and the external phase. However, it doesn't explicitly explain what diffusivity is in terms. To provide an explanation, diffusivity can be described as the proportionality constant that quantifies how fast a substance spreads through a given medium under the influence of a concentration gradient.

🔍 Source documents:

- `docs/KB/wiki/regulations/P100BCMB.md`

⏱️ **Response time**: Typically **between 10 and 40 seconds**, depending on your **local GPU** and **model size**.

> ⚠️ If you are running on CPU or with limited VRAM, expect longer runtimes and adjust batch size or chunking as needed.

🚀 Why It Matters

- A **local model** (Mistral) with <8 GB VRAM can perform advanced reasoning when grounded with curated data.
- The KB is **versionable, auditable, and extendable** — ideal for industrial compliance, safety, and design workflows.
- The same infrastructure can be used to run **SAG pipelines** where simulation results are also integrated into LLM prompts.

🌐 Related Concepts

Concept	Description
RAG	Retrieval-Augmented Generation, used here for regulatory and scientific QA
SAG	Simulation-Augmented Generation, a GS-native variant where simulation output augments LLM reasoning
ChromaDB	Vector store used to persist embeddings and ensure consistent indexing
Ollama	Lightweight local server to run open-source LLMs (Mistral, LLaMA2, etc.)

🧰 Advanced Options

- You can add **confidential regulatory PDFs** (converted to Markdown), internal compliance notes, or even `SFPPy` simulation logs to your KB.
- You can fine-tune retrieval using metadata or chunk size if needed.
- You can integrate simulation outputs (`R.__doc__` , `R.summary()` , etc.) into the KB for **SAG reasoning**.

📊 Minimal Footprint, Maximal Insight

This example demonstrates that **high-value insights** can be obtained:

- Without exposing data to the cloud.
- With open tools and standardized formats.
- Within a reproducible, inspectable AI-assisted workflow.

> 🔍 *RAG and SAG are complementary. RAG grounds the question. SAG integrates the answer.*
> With SFPPy, both can be achieved within your local infrastructure.

⚙️ Installation Instructions

To use **Example 5** (Retrieval-Augmented QA using a local LLM), you need to set up a Python environment with a clean stack of AI and vector libraries. We **recommend using** `conda` **for the base environment** and `pip` to install required packages.

🔄 Why mix `conda` and `pip`?

- `conda` is ideal for managing **Python versions**, **dependencies**, and **C/C++ libraries** (e.g., `numpy`, `scipy`, `libstdc++`, `openssl`, etc.).
- `pip` is better suited for **rapidly evolving Python libraries**, especially in the **LLM and vector space** (`llama-index`, `chromadb`, etc.).
- This hybrid approach ensures **maximum compatibility and reproducibility**, especially across GPU and CPU machines.

🧪 Step-by-Step Setup

```
conda create -n llm-stack python=3.10 -y
conda activate llm-stack
pip install llama-index llama-index-embeddings-huggingface \
            llama-index-llms-ollama llama-index-vector-stores-chroma \
            llama-index-vector-stores sentence-transformers chromadb
```

Optional — for advanced ingestion of PDF/HTML/Office files:

```
pip install unstructured
```

You can now run `example5.py` in this environment.

🧠 Optional: Run from Spyder IDE

If you use **Spyder** and want to run `example5.py` interactively, follow these extra steps:

```
pip install ipykernel
python -m ipykernel install --user --name llm-stack --display-name "Python (llm-stack)"
```

Then in **Spyder**:

- Open a new console via `Consoles → New Console → Python (llm-stack)`
- Load or run `example5.py` from the new kernel

> 📌 Spyder must be restarted if the kernel list was already open.

🧪 Test Your Installation

Run this minimal script to confirm everything works:

```
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama

documents = SimpleDirectoryReader(input_dir="./docs/KB", recursive=True).load_data()
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
index = VectorStoreIndex.from_documents(documents, embed_model=embed_model)
llm = Ollama(model="mistral")
query_engine = index.as_query_engine(llm=llm)
response = query_engine.query("What is migration?")
print(response)
```

> ✅ If you see a coherent answer with source references, the installation is successful.
